

**Taller de Investigación: Métodos Abreviados y Sintaxis Comprimidas en
Java**

Ficha: 3407799

Instructor: Carlos Julio

Tecnólogo en Análisis y Desarrollo de Software

Aprendiz: Steven Pérez Pólania

SENA Sede de la Industria, la empresa y los servicios

Contenido

1. Clase Optional	3
2. Set.of().....	4
3. Map.of().....	4
4. Operador Diamond (<>)......	5
5. String.format().....	6
6. Operador Ternario (?:)	6
7. Inferencia de Tipos con var	7
8. Expresiones Lambda	7
9. Referencias a Métodos (::)	8
10. API Streams.....	8
11. Switch Expression.....	9
12. Text Blocks.....	9
13. Records.....	10
14. Pattern Matching con instanceof	10
15. Ciclo For-each.....	11
16. Hilos Virtuales (Virtual Threads - Proyecto Loom).....	11
17. Colecciones Secuenciales (Sequenced Collections)	12
18. Variables y Patrones Innominados (Unnamed Variables)	12
19. Bloques Try-with-resources con Variables Efectivamente Finales.....	13
20. Método Objects.requireNonNullElse().....	14

Parte 1: 15 Temas Seleccionados

1. Clase Optional

- Explicación: Contenedor introducido en Java 8 para representar la presencia o ausencia de un valor, evitando el uso directo de referencias nulas.
- Ventajas: Elimina los molestos NullPointerException y hace explícita la opcionalidad en las firmas de los métodos.
- Desventajas: Añade un ligero costo de envoltura en memoria; no debe usarse para campos de clase o parámetros de métodos.
- Cuándo utilizarlo: Como tipo de retorno en métodos que legítimamente pueden no encontrar un resultado.

- Comparación:

Tradicional:

```
if (usuario != null) {  
    return usuario.getNombre();  
} else {  
    return "Anónimo";  
}
```

Abreviada:

```
return optionalUsuario.map(Usuario::getNombre).orElse("Anónimo");
```

2. Set.of()

- Explicación: Método de factoría estática (Java 9) para instanciar conjuntos (Set) inmutables que no permiten elementos duplicados ni nulos.
- Ventajas: Sintaxis limpia de una sola línea, alta eficiencia en memoria y protección nativa contra modificaciones.
- Desventajas: Arroja `IllegalArgumentException` en tiempo de diseño/ejecución si se intenta ingresar un elemento duplicado.
- Cuándo utilizarlo: Para definir colecciones de elementos únicos y fijos como roles de usuario o configuraciones estáticas.

- Comparación:

Tradicional:

```
Set<String> roles = new HashSet<>();  
roles.add("ADMIN");  
roles.add("USER");  
roles = Collections.unmodifiableSet(roles);
```

Abreviada:

```
Set<String> roles = Set.of("ADMIN", "USER");
```

3. Map.of()

- Explicación: Método de factoría estática para inicializar diccionarios (Map) inmutables de hasta 10 parejas de clave-valor en una sola expresión.

- Ventajas: Sintaxis compacta que elimina bloques repetitivos de inicialización pesada.
- Desventajas: Limitado estrictamente a un máximo de 10 llaves (para más se debe usar Map.ofEntries) e inmutable.
- Cuándo utilizarlo: Al crear mapas de traducción simples, diccionarios fijos de constantes o configuraciones clave-valor breves.
- Comparación:

Tradicional:

```
Map<String, String> config = new HashMap<>();
config.put("db", "mysql");
config.put("port", "3306");
```

Abreviada:

```
Map<String, String> config = Map.of("db", "mysql", "port", "3306");
```

4. Operador Diamond (<>)

- Explicación: Sintaxis que permite omitir los argumentos de tipo genérico en el constructor cuando el compilador puede inferirlos del lado izquierdo.
- Ventajas: Reduce la redundancia visual manteniendo la seguridad de tipos estática intacta.
- Desventajas: No se puede usar con clases anónimas internas de forma directa en versiones previas a Java 9.
- Cuándo utilizarlo: En cualquier instanciación estándar de colecciones u objetos basados en genéricos.
- Comparación:

Tradicional:

```
List<Map<String, List<Integer>>> lista = new ArrayList<Map<String,
List<Integer>>>());
```

Abreviada:

```
List<Map<String, List<Integer>>> lista = new ArrayList<>();
```

5. String.format()

- Explicación: Método de utilidad para construir cadenas de texto con formato utilizando marcadores de posición basados en el estilo clásico de C (printf).
- Ventajas: Mejora drásticamente la legibilidad frente a concatenaciones interminables con el operador '+'.
- Desventajas: Es internamente más lento debido al parseo de expresiones regulares y al boxing de argumentos.
- Cuándo utilizarlo: Para construir mensajes dinámicos legibles, reportes estructurados o logs informativos.
- Comparación:

Tradicional:

```
String msg = "El usuario " + nombre + " tiene " + edad + " años y saldo de $" + saldo;
```

Abreviada:

```
String msg = String.format("El usuario %s tiene %d años y saldo de $%.2f", nombre, edad, saldo);
```

6. Operador Ternario (?:)

- Explicación: Evaluación abreviada de condiciones booleanas para retornar un valor directo.
- Ventajas: Compacta estructuras condicionales simples a una sola línea.
- Desventajas: Se vuelve confuso si se anida o si incluye expresiones con efectos secundarios.
- Cuándo utilizarlo: Asignaciones rápidas de variables bajo condiciones binarias.
- Comparación:

Tradicional:

```
int max; if (a > b) { max = a; } else { max = b; }
```

Abreviada:

```
int max = (a > b) ? a : b;
```

7. Inferencia de Tipos con var

- Explicación: Omisión del tipo explícito en declaraciones locales dejando que el compilador lo deduzca por el contexto inmediato.
- Ventajas: Limpia firmas visuales complejas, agilizando la escritura de código iterativo.
- Desventajas: Oculta el tipo real si los nombres de los métodos del lado derecho no son claros.
- Cuándo utilizarlo: Variables locales con inicialización clara y evidente.
- Comparación:

Tradicional:

```
BufferedReader lector = new BufferedReader(new  
    FileReader("archivo.txt"));
```

Abreviada:

```
var lector = new BufferedReader(new FileReader("archivo.txt"));
```

8. Expresiones Lambda

- Explicación: Bloques funcionales anónimos que permiten pasar comportamiento puro como si fuera un parámetro.
- Ventajas: Habilita la programación funcional reduciendo las boilerplate classes anónimas.
- Desventajas: Dificulta la lectura de stack traces durante excepciones complejas.
- Cuándo utilizarlo: Al alimentar filtros, ordenamientos o ejecuciones asíncronas en APIs estructuradas.
- Comparación:

Tradicional:

```
new Thread(new Runnable() {  
    @Override public void run() { System.out.println("Ejecutando"); }  
}).start();
```

Abreviada:

```
new Thread(() -> System.out.println("Ejecutando")).start();
```

9. Referencias a Métodos (::)

- Explicación: Atajo sintáctico para invocar métodos existentes de manera directa sin evaluar argumentos repetitivos.
- Ventajas: Concisión absoluta cuando una lambda solo actúa como puente hacia otro método.
- Desventajas: No permite realizar transformaciones o adaptaciones sobre los parámetros pasados.
- Cuándo utilizarlo: Pasar flujos limpios a lambdas puras de mapeo o impresión.
- Comparación:

Tradicional:

```
lista.stream().map(s -> s.toUpperCase());
```

Abreviada:

```
lista.stream().map(String::toUpperCase);
```

10. API Streams

- Explicación: Canalizaciones operacionales para el procesamiento declarativo y funcional de datos masivos.
- Ventajas: Elimina bucles 'for' complejos y acumuladores temporales manuales.
- Desventajas: Puede inducir sobrecostos leves en listas extremadamente reducidas.
- Cuándo utilizarlo: Transformaciones, mapeos y filtrados complejos de colecciones de negocio.
- Comparación:

Tradicional:

```
int suma = 0;  
for(int n : numeros) { if(n % 2 == 0) suma += n; }
```

Abreviada:

```
int suma = numeros.stream().filter(n -> n % 2 ==  
0).mapToInt(Integer::intValue).sum();
```

11. Switch Expression

- Explicación: Evolución sintáctica del condicional switch que retorna un valor directo por medio de flechas funcionales.
- Ventajas: Previene bugs de omisión de break y garantiza exhaustividad de casos.
- Desventajas: Requiere abarcar de manera obligatoria todas las opciones de datos del tipo evaluado.
- Cuándo utilizarlo: Mapeos directos de estados o enums a variables funcionales.
- Comparación:

Tradicional:

```
String desc;  
switch(codigo) { case 1: desc="Activo"; break; default: desc="Inactivo"; }
```

Abreviada:

```
String desc = switch(codigo) { case 1 -> "Activo"; default -> "Inactivo"; };
```

12. Text Blocks

- Explicación: Cadenas multilínea con tres comillas que conservan la indentación original nativamente.
- Ventajas: Evita el escape continuo de caracteres de control como comillas y saltos de línea.
- Desventajas: Requiere atención con la sangría de cierre para evitar espacios fantasmas.
- Cuándo utilizarlo: Incrustación de código SQL, JSON, HTML o XML estructurado dentro del código.
- Comparación:

Tradicional:

```
String html = "<html>\n<body>\n<p>Hola</p>\n</body>\n</html>";
```

Abreviada:

```
String html = ""  
    <html>  
    <body>  
    <p>Hola</p>  
    </body>  
    </html>  
    "";
```

13. Records

- Explicación: Estructuras de datos compactas e inmutables que generan implícitamente getters, constructores y métodos estructurales.
- Ventajas: Ideal para la creación instantánea de objetos de transferencia de datos (DTO).
- Desventajas: No permite heredar de otras clases convencionales ni alterar campos tras su creación.
- Cuándo utilizarlo: Modelado de payloads de APIs o contenedores de base de datos puros.
- Comparación:

Tradicional:

Clase Verbosa POJO con constructor, getters, hashCode, equals y toString manuales.

Abreviada:

```
public record Producto(int id, String nombre) {}
```

14. Pattern Matching con instanceof

- Explicación: Verificación de tipo e inicialización instantánea de una variable de casting seguro en un solo paso.
- Ventajas: Limpia los bloques repetitivos de validación y subsecuente casteo manual.
- Desventajas: El ámbito del objeto resultante se limita a la ramificación lógica de cumplimiento.

- Cuándo utilizarlo: Operaciones polimórficas dinámicas en métodos utilitarios como equals.

- Comparación:

Tradicional:

```
if(obj instanceof Double) { Double d = (Double)obj; num = d.intValue(); }
```

Abreviada:

```
if(obj instanceof Double d) { num = d.intValue(); }
```

15. Ciclo For-each

- Explicación: Estructura optimizada para recorrer colecciones secuenciales e iterables sin punteros manuales.

- Ventajas: Evita errores de desfase de índices fuera de límites.

- Desventajas: Impide la remoción directa de elementos durante la marcha sin iterador explícito.

- Cuándo utilizarlo: Lectura secuencial homogénea completa de arreglos o colecciones.

- Comparación:

Tradicional:

```
for (int i = 0; i < datos.length; i++) { procesar(datos[i]); }
```

Abreviada:

```
for (int dato : datos) { procesar(dato); }
```

Parte 2: Investigación Adicional (5 Nuevos Temas Autónomos Diferentes)

16. Hilos Virtuales (Virtual Threads - Proyecto Loom)

- Explicación: Hilos ultraligeros administrados por la JVM (Java 21) en lugar del sistema operativo, permitiendo ejecutar millones de hilos concurrentes simultáneamente.

- **Ventajas:** Escalabilidad masiva para servidores web sin cambiar el paradigma imperativo secuencial clásico por un modelo reactivo asíncrono complejo.
- **Desventajas:** No están diseñados para tareas intensivas de cómputo de CPU puro, sino para operaciones de alta E/S (bloqueos de red o base de datos).
- **Cuándo utilizarlo:** En backend corporativo que atiende solicitudes concurrentes bloqueantes pesadas de red o API.
- **Ejemplo Funcional:**

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    executor.submit(() -> { System.out.println("Tarea en hilo virtual
seguro"); });
}
```

17. Colecciones Secuenciales (Sequenced Collections)

- **Explicación:** Nueva jerarquía de interfaces unificada en Java 21 (como SequencedCollection, SequencedSet) que define un orden explícito de principio a fin para colecciones con métodos predecibles.
- **Ventajas:** Proporciona acceso directo y estandarizado al primer y último elemento de manera uniforme sin recurrir a trucos basados en iteradores de bajo nivel.
- **Desventajas:** Requiere la migración de dependencias y actualización mental de métodos tradicionales como get(0) o iteraciones pesadas.
- **Cuándo utilizarlo:** Al manipular estructuras ordenadas donde el acceso a los extremos (frente/cola) sea constante.
- **Ejemplo Funcional:**

```
LinkedHashSet<String> nombres = new LinkedHashSet<>();
nombres.addFirst("Steveen");
String ultimo = nombres.getLast();
```

18. Variables y Patrones Innominados (Unnamed Variables)

- **Explicación:** Introducción del guion bajo (_) en Java 21 para declarar variables locales, parámetros o excepciones que deben existir por sintaxis pero que no se utilizarán en el bloque.

- **Ventajas:** Mejora la claridad visual del código al explicitar que una variable es ignorada a propósito, evitando advertencias estáticas del compilador.
- **Desventajas:** Puede causar confusión a programadores junior no familiarizados con lenguajes funcionales o de tipado avanzado.
- **Cuándo utilizarlo:** En capturas de excepciones donde no se usa el objeto del error o en iteraciones de mapas donde solo importa la llave.
- **Ejemplo Funcional:**

```
try {
    int numero = Integer.parseInt("texto");
} catch (NumberFormatException _) {
    System.out.println("Entrada inválida detectada de manera limpia.");
}
```

19. Bloques Try-with-resources con Variables Efectivamente Finales

- **Explicación:** Mejora en la estructura try-with-resources (Java 9) que permite pasar una variable de recurso ya inicializada directamente dentro del paréntesis sin rediseñarla ni redeclararla localmente.
- **Ventajas:** Mantiene limpia la firma del bloque de control reduciendo el anidamiento y duplicidad de nombres.
- **Desventajas:** La variable externa obligatoriamente debe ser final o efectivamente final, impidiendo reasignaciones antes o después del bloque.
- **Cuándo utilizarlo:** Al gestionar cierres automáticos de streams o sockets inicializados en métodos superiores o de fábrica externos.
- **Ejemplo Funcional:**

```
var conexion = DriverManager.getConnection("jdbc:mysql://localhost/db");
try (conexion) {
    // Procesar datos de infraestructura con cierre garantizado
    automáticamente
}
```

20. Método `Objects.requireNonNullElse()`

- Explicación: Método estático utilitario de validación (Java 9) que evalúa si un objeto es nulo; si lo es, retorna inmediatamente un objeto por defecto alternativo provisto en el segundo argumento.
- Ventajas: Sintaxis compacta para asignaciones seguras a prueba de nulos que reemplaza bloques if convencionales o lógica ternaria duplicada.
- Desventajas: Evalúa el segundo argumento inmediatamente, por lo que si es una llamada a método costoso, se ejecutará siempre (no es perezoso).
- Cuándo utilizarlo: Al inicializar valores predeterminados seguros en constructores o inicializadores de negocio.
- Ejemplo Funcional:

```
    this.descripcion = Objects.requireNonNullElse(inputDesc, "Sin descripción  
    asignada por defecto");
```

Referencias Bibliográficas

- Oracle Corporation. (2023). Java Language Specification (Java SE 21 Edition). Oracle Technology Network.
- Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley Professional.
- Evans, B., & Warburton, R. (2022). Java in a Nutshell (8th ed.). O'Reilly Media.